RTDIP x AMOS

Design Documentation

Overview

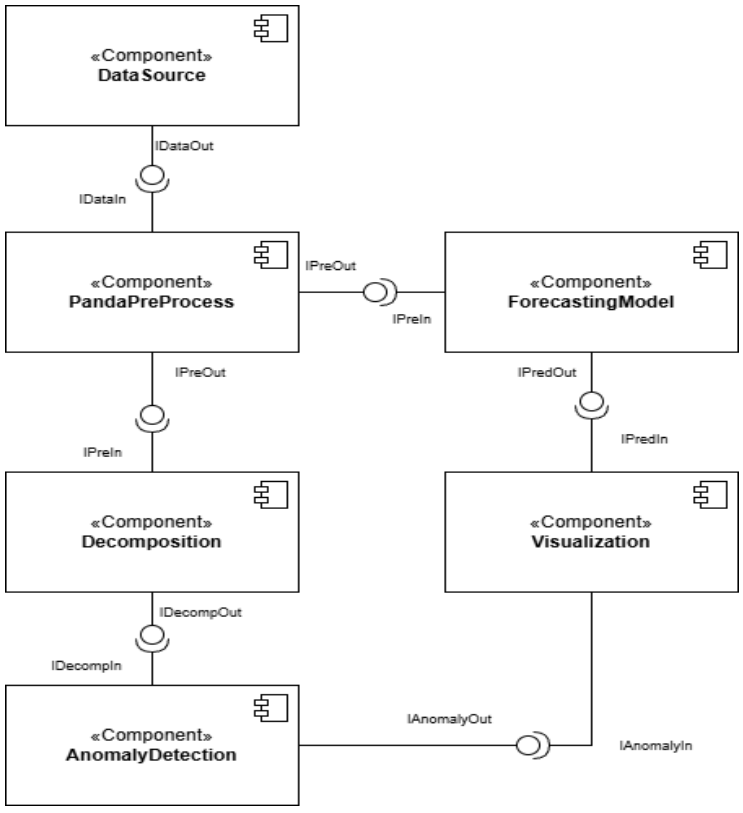
The RTDIP Time Series Forecasting project provides a modular and extensible pipeline for processing, forecasting, and analyzing time-series data. The system is designed around clearly defined components and interface-driven contracts, enabling flexible integration of preprocessing, forecasting, decomposition, anomaly detection, and visualization techniques. By enforcing separation of concerns and centralized orchestration, the architecture supports maintainability, testability, and easy extension with new models or detection strategies.

1. System Components

The RTDIP Time Series Forecasting system is composed of a set of loosely coupled components, each responsible for a specific stage in the data processing and analysis pipeline. Components communicate exclusively through well-defined interfaces and are orchestrated by a central execution script, ensuring high cohesion, low coupling, and extensibility.

1.1 High-Level Architecture

Figure 1 illustrates the high-level architecture of the RTDIP forecasting pipeline, showing the flow of data between system components.



1.2 DataSource

A mock component that simulates loading time-series data from real RTDIP sources such as Azure Delta tables, Blob Storage, or other cloud providers.

1.3 PandaPreProcess

A component that groups all Pandas-based preprocessing steps, including data cleaning, feature engineering, timestamp handling, encoding, and outlier detection.

1.4 ForecastingModel

A component that groups all forecasting model implementations. In this project, it wraps the AutoGluon time-series predictor for model training and inference.

1.5 Decomposition

A component that performs time-series decomposition to extract trend, seasonal, and residual components. It supports STL decomposition, MSTL decomposition, and automated period calculation.

1.6 AnomalyDetection

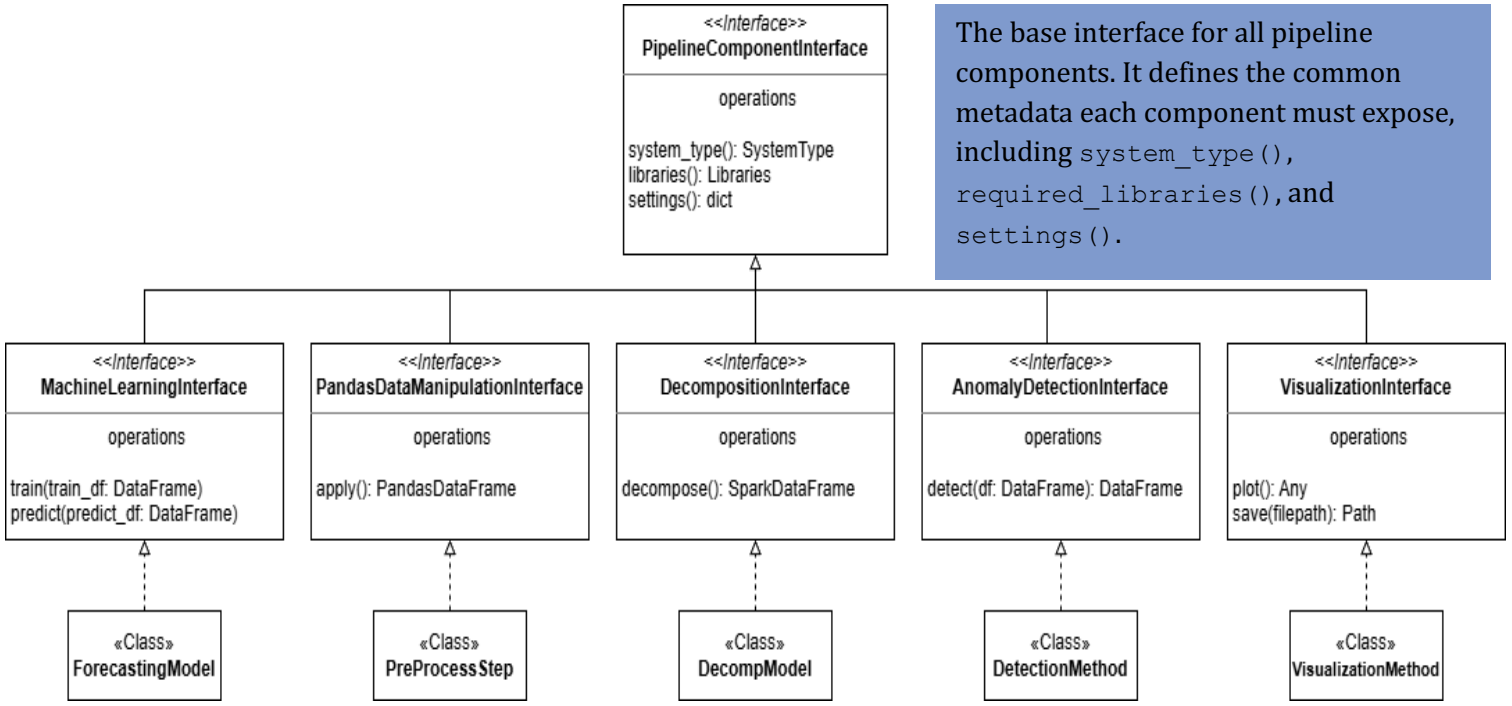
A component that identifies anomalies in time-series data using statistical methods. It implements the MAD (Median Absolute Deviation) and the IQR (Interquartile Range) approaches with configurable scoring strategies.

1.7 Visualization

A component that consumes model predictions and produces plots or charts for evaluation and result analysis.

2.Interfaces

Figure 2: UML Class Diagram of Pipeline Component Interfaces and Implementations



The base interface for all pipeline components. It defines the common metadata each component must expose, including `system_type()`, `required_libraries()`, and `settings()`.

Specialized interface for all machine-learning components. It defines the methods train() and predict() that every forecasting model must implement.

Specialized interface for all Pandas-based preprocessing components. It defines the apply() method used to transform a DataFrame.

Specialized interface for time series decomposition components. It defines the decompose() method that returns trend, seasonal, and residual

Specialized interface for anomaly detection components. It defines the detect() method that returns a DataFrame with anomaly flags.

Specialized interface that defines methods to create and save plots for model predictions and analysis.

3. Anomaly Detection

3.1 MAD Anomaly Detection

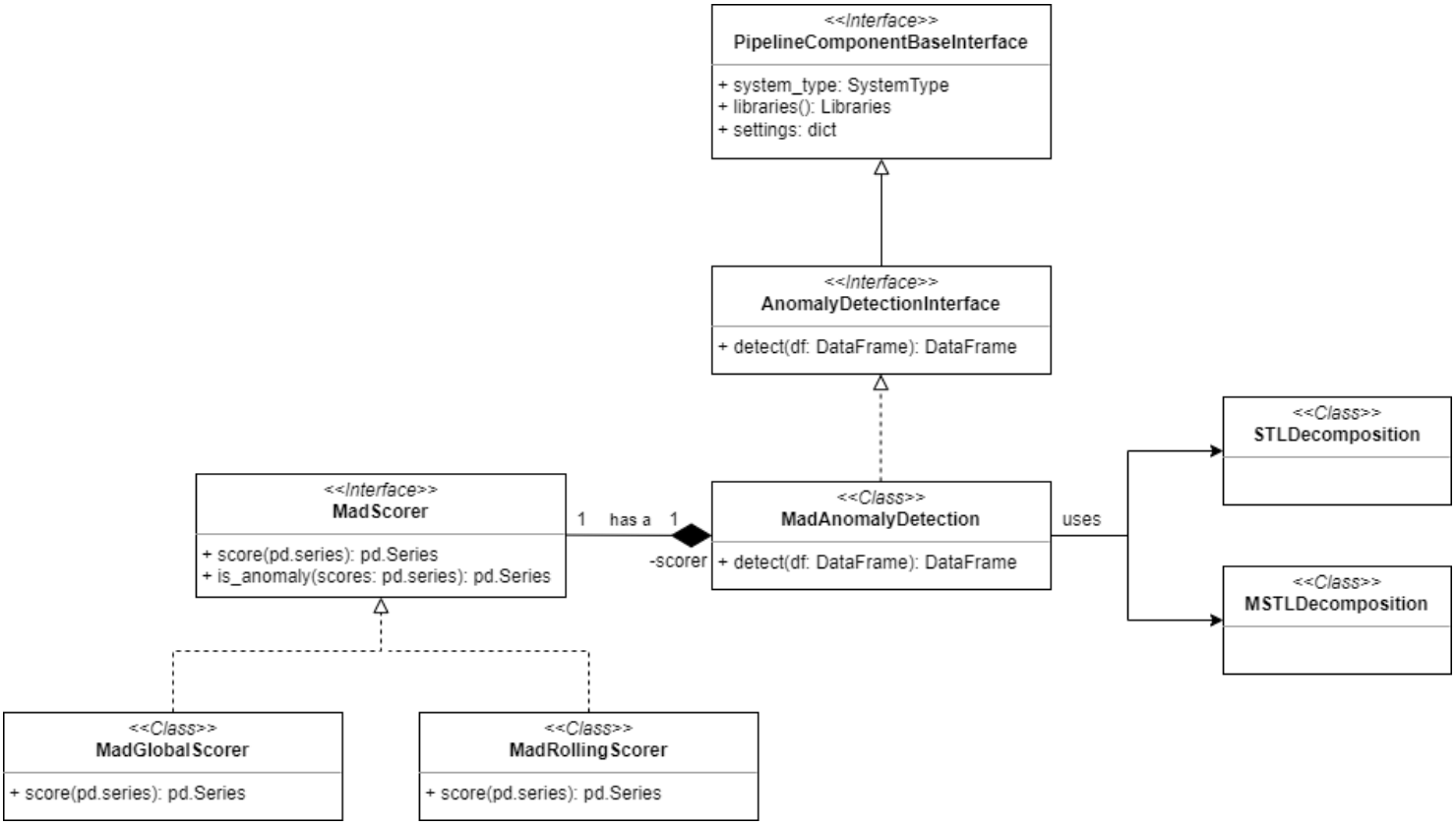


Figure 3: Class Diagram of MAD Anomaly Detection and Decomposition Components.

The design separates **pipeline-facing components** from **internal algorithm strategies**.

AnomalyDetectionInterface extends PipelineComponentBaseInterface and therefore defines the **pipeline-facing contract** for anomaly detection components.

MadAnomalyDetection is one **concrete implementation** of this pipeline component contract shown in the diagram.

The remaining elements serve different roles:

- MadScorer is a scoring strategy interface used by MadAnomalyDetection via composition. Implementations such

as MadGlobalScorer and MadRollingScorer encapsulate alternative MAD-based scoring behaviors.

- STLDecomposition and MSTLDecomposition are **standalone pipeline components.**. Wrapper/adaptor logic is used so they can be consumed consistently by MadAnomalyDetection.

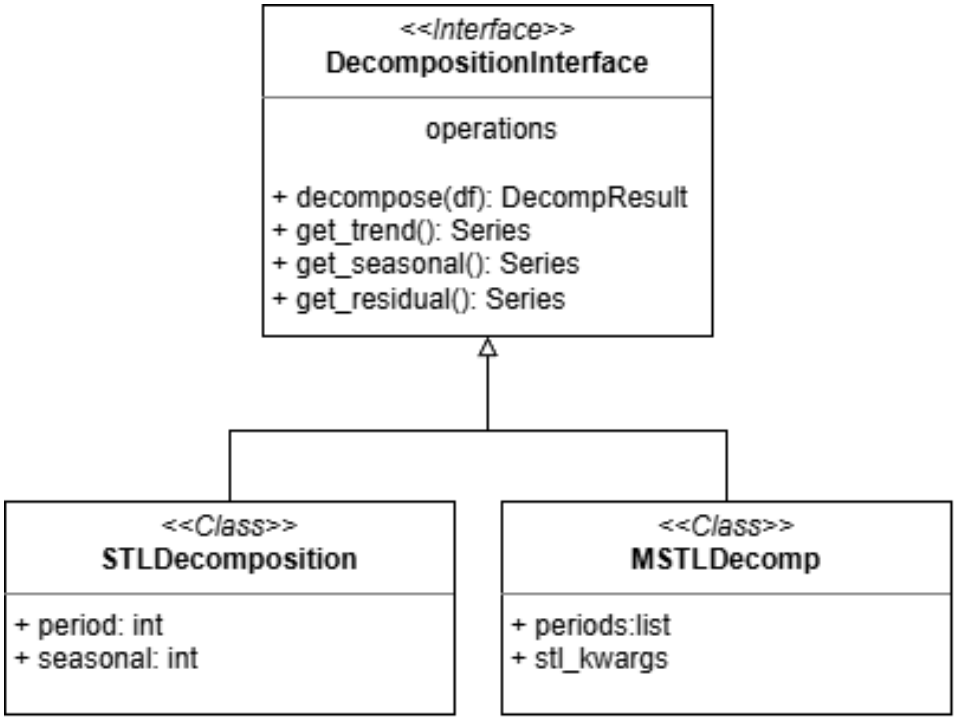
3.2 IQR Anomaly Detection

The IQR anomaly detection component is designed as a **modular, interface-compliant detector** within the RTDIP framework. It implements the **AnomalyDetectionInterface**, ensuring consistent interaction with the rest of the pipeline and interchangeable use alongside other detectors like MAD.

Key design elements:

- **Interface Compliance:**
  The IQR detector exposes a `detect()` method that accepts a DataFrame and returns a DataFrame augmented with anomaly flags, adhering strictly to the `AnomalyDetectionInterface` contract.
- **Global vs. Rolling Detection:**
  The design supports two modes:
  - *Global IQR*: Calculates quartiles and thresholds over the entire time series at once, suitable for detecting extreme, global outliers.
  - *Rolling IQR*: Computes quartiles and thresholds within a sliding time window, allowing the detector to adapt to local changes and detect contextual anomalies.
- **Parameterization:**
  The sensitivity factor k(multiplying the interquartile range) and the rolling window size are configurable parameters, enabling users to tune detection behavior according to data characteristics.
- **Stateless and Transparent Logic:**
  The detection logic is purely statistical and does not require model training. This makes it lightweight and easy to interpret, facilitating quick anomaly flagging without complex dependencies.
- **Integration with Scorers (optional):**
  Although simpler than MAD's scoring architecture, the IQR detector can be extended or paired with scoring mechanisms for threshold adjustment or confidence scoring.
- **Reusability and Extensibility:**
  The component is designed for easy maintenance and future extension, allowing additional statistical detectors to follow the same interface and plug seamlessly into the RTDIP pipeline.

4. Time Series Decomposition



Seasonal-Trend decomposition using LOESS. Separates a time series into trend, seasonal, and residual components for a single seasonal period.

Multiple Seasonal-Trend decomposition. Handles time series with multiple seasonal patterns (e.g., daily and weekly seasonality). Includes automated period calculation for automatic detection of seasonal periods.

5. Data Manipulation

The design separates pipeline-facing components from cross-cutting validation concerns. PipelineComponentBaseInterface is the root interface for all pipeline components, defining system type, library dependencies, and settings. Three specialized interfaces extend this base:

- DataManipulationBaseInterface defines filter_data() for Spark-based data transformation.
- PandasDataManipulationBaseInterface defines apply() for Pandas-based local processing.
- MonitoringBaseInterface defines check() for data quality validation.

Spark Data Manipulation classes (KSigmaAnomalyDetection, MissingValueImputation, FlatlineFilter, etc.) are concrete implementations of DataManipulationBaseInterface for distributed filtering.

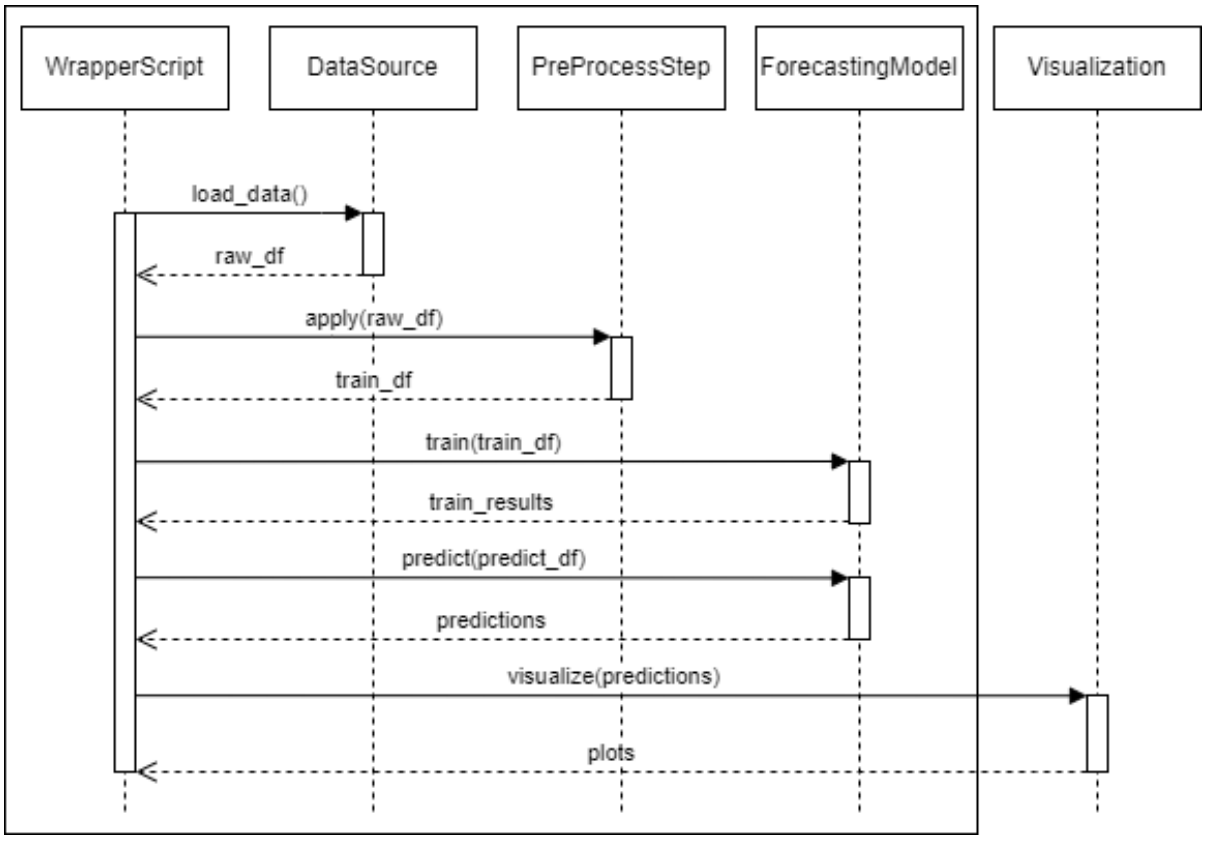
NormalizationBaseClass is an abstract base class using the template method pattern. Subclasses (NormalizationMean, NormalizationMinMax, NormalizationZScore) provide specific normalization algorithms.

Pandas Data Manipulation classes (MADOutlierDetection, DatetimeFeatures, etc.) are concrete implementations of PandasDataManipulationBaseInterface for single-machine processing.

Monitoring classes (FlatlineDetection, CheckValueRanges, etc.) are concrete implementations of MonitoringBaseInterface that return quality check results rather than filtered data.

6. General Forecasting Sequence

Figure 4: sequence diagram that shows the execution flow of the RTDIP-style forecasting pipeline script.



A wrapper script orchestrates the entire pipeline. It first calls the DataSource component to load the raw dataset using load_data(). The DataSource returns a raw dataframe to the wrapper script.

The wrapper script then sends this raw dataframe to the PreProcessStep component by calling apply(raw_df). The preprocessing component returns a cleaned and transformed dataframe.

Next, the wrapper script calls the ForecastingModel component. It passes the preprocessed dataframe to train(train_df). After training, it calls predict(predict_df) to obtain the forecast results. The model returns the predictions to the wrapper.

Finally, the wrapper script calls the Visualization component through visualize(predictions). The visualization component generates plots and returns them to the wrapper script.

The components never call each other directly. All communication is coordinated by the wrapper script. The data flows through the pipeline in the order: DataSource -> PreProcessStep -> ForecastingModel -> Visualization.

Note: Some steps (eg. loading the data from azure) were not included in the wrapper demo